

Docker Images, Containers and Compose Files

What each image contains, which containers run from it, how the containers work together, what every compose file starts, and which one to use when.

Covers the three Dockerfiles in `deploy/` (four images), the seven compose files, and the scripts that start containers with plain `docker run` (AWS, EC2, LAN lab). Image sizes are as listed by `docker images` on the laptop (Docker Desktop, WSL2). All paths are relative to the repository root.

Contents

1. The big picture
2. The images
3. What runs inside the containers
4. The compose files, one by one
5. How the containers work together
6. Deployments without compose (docker run scripts)
7. Which one to use when
8. Build, save and move images (air gap)
9. Gotchas
10. Quick reference

1. The big picture

A job on this platform needs three kinds of software, and the images keep them apart:

- **Spark:** Java 17 and Spark 3.5.1 for the master, the workers and the executors; Python 3.11 and pyspark for the driver and the Python workers.
- **The models' runtime:** torch 2.6 with its CUDA 12.6 runtime libraries, plus each model's own dependencies (for NER: GLiNER, transformers, sentencepiece, py3langid, tesseract OCR with 11 language packs, poppler).
- **The code and the weights:** the platform code (inference/, models/, submit_*.py) and the model weights. The code is baked into the images and usually also bind-mounted over them. The weights are mounted, or fetched from the HDFS model store.

28. Docker images - what is built from what

Three Dockerfiles, four images; arrows = FROM / build target. Sizes as listed by docker images on the laptop

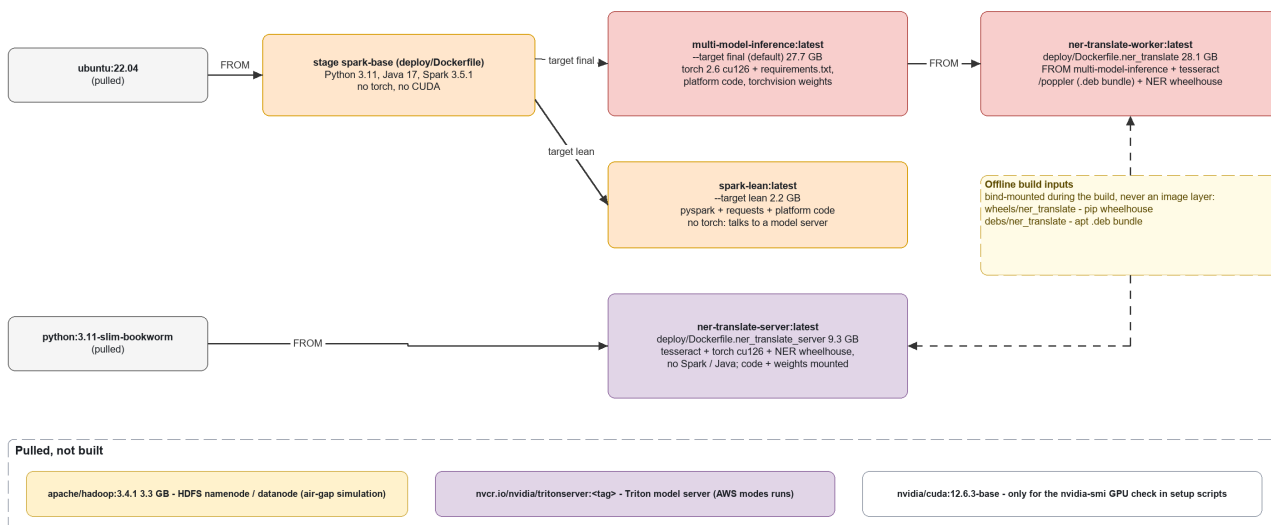


Figure 1 - the image family: three Dockerfiles, four built images, three pulled ones

image	built by	contains	leaves out	size	runs as
multi-model-inference:latest	deploy/Dockerfile --target final	Ubuntu 22.04, Python 3.11, Java 17, Spark 3.5.1, torch 2.6 + torchvision (cu126), requirements.txt, platform code, ResNet18 / MobileNetV3 / EfficientNet-B0 weights	the CUDA driver (comes from the host)	27.7 GB	Spark master, workers and driver for tensor models; base of ner-translate-worker
spark-lean:latest	deploy/Dockerfile --target lean	Spark 3.5.1, Python 3.11, pyspark, requests, platform code	torch, CUDA, any model dependency	2.2 GB	Spark master / worker / driver in waiter-kitchen (service) mode
ner-translate-worker:latest	deploy/Dockerfile.ner_translate	everything in multi-model-inference + tesseract / poppler (from a .deb bundle) + the NER pipeline's Python packages (from a wheelhouse) + apply_wheels_hotfix.sh	-	28.1 GB	Spark master / worker when the executors run the NER pipeline themselves (cluster mode)
ner-translate-server:latest	deploy/Dockerfile.ner_translate_server	Python 3.11 slim, tesseract + poppler, torch 2.6 cu126, NER packages, FastAPI / uvicorn	Spark, Java, the code and the weights (all mounted from models/)	9.3 GB	the kitchen: POST /predict on :8000
apache/hadoop:3.4.1	pulled	Hadoop 3.4.1 (HDFS)	-	3.3 GB	HDFS namenode / datanode in the air-gap simulation
nvcr.io/nvidia/tritonserver:<tag>	pulled	NVIDIA Triton Inference Server	-	-	model server for the Triton mode (AWS modes runs)
nvidia/cuda:12.6.3-base	pulled	CUDA base + nvidia-smi	-	-	GPU visibility check in setup scripts only

Three architectures come out of these images. Every NER compose file is one of B or C; the tensor-model files are A:

29. Three ways the containers run a model - and which compose files build each

[image] in brackets; host ports as published by the compose files

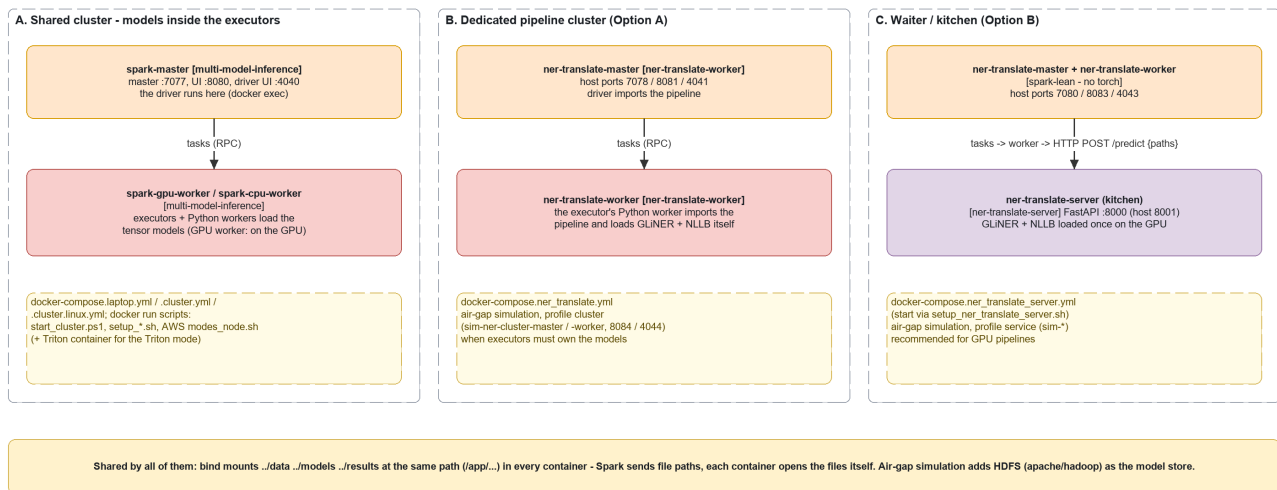


Figure 2 - the three ways containers run a model

2. The images

2.1 deploy/Dockerfile - three stages, two images

stage	FROM	adds	image built from it
spark-base	ubuntu:22.04	Python 3.11 (deadsnakes PPA), Java 17 JRE, Spark 3.5.1 in /opt/spark, PYSPARK_PYTHON=python, WORKDIR /app	none on its own - the shared foundation
final	spark-base	torch 2.6.0 + torchvision 0.21.0 (cu126 wheels), requirements.txt, the platform code (copied from a bind mount, skipping wheels/, debs/ and .git), results/, three torchvision checkpoints; CMD run_benchmark.py	multi-model-inference:latest
lean	spark-base	pyspark 3.5.1, requests, the platform code; CMD bash	spark-lean:latest

The default build target is lean, not final. lean is the last stage in the file, and Docker builds the last stage when no `--target` is given (`docker buildx build --call=targets` lists lean (default)). So `docker build -t multi-model-inference:latest -f deploy/Dockerfile .` - the command in `setup_master.sh`, `setup_gpu_worker.sh`, `setup_and_run_gpu.sh`, `run_gpu_benchmarks.ps1`, `run_benchmarks_cloud.ps1`, the CDK user data, `build_and_test_ner_translate_gpu.sh` and the build section of `docker-compose.yml` - produces the torch-less lean image under the multi-model-inference name. Only `aws_campaign.sh` and `modes_node.sh` pass `--target final`. Until the stages are reordered (final last), always build with `--target final`.

Why there is no nvidia/cuda base image: the cu126 torch wheels bring their own CUDA runtime libraries (cuBLAS, cuDNN, cuFFT, ...) as pip packages, and the GPU driver (libcudart plus the device files) is injected at run time by the NVIDIA Container Toolkit. A CUDA base image would only duplicate several GB of the same libraries.

2.2 deploy/Dockerfile.ner_translate - the NER image for in-process execution

FROM multi-model-inference:latest, then only this pipeline's own dependencies, installed **offline**:

- tesseract-ocr with the 11 OCR language packs, and poppler-utils, installed with `dpkg -i` from `debs/ner_translate` (built by `deploy/scripts/build_ner_translate_debs.sh`). `dpkg` runs twice because it does not order packages: the first pass unpacks, the second configures.
- The pipeline's Python packages, installed with `pip install --no-index --find-links` from `wheels/ner_translate` (built by `deploy/scripts/build_ner_translate_wheelhouse.sh`). The build fails loudly if a pinned version is missing rather than resolving a different one.
- Both bundles are **bind-mounted** into their RUN step, never COPY'd: a COPY layer would stay in docker save output even after a later `rm`.
- `apply_wheels_hotfix.sh` runs at container start and installs any .whl dropped into the /wheels-hotfix mount. It is a way to patch one dependency on an already-deployed air-gapped node without rebuilding. Put the same wheels on every node, or the driver and the workers end up with different versions.

2.3 deploy/Dockerfile.ner_translate_server - the kitchen

FROM python:3.11-slim-bookworm. It has no Spark and no Java: the kitchen never talks to Spark, it only answers HTTP. It installs tesseract + poppler, torch 2.6 cu126 (the same version as the Spark-side image, so results match) and the NER wheelhouse. It contains **no application code and no weights**: both come from the mounted models/ folder. CMD is `uvicorn models.pipelines.ner_translate.serve:app --host 0.0.0.0 --port 8000`, and it exposes port 8000.

2.4 Pulled images

- **apache/hadoop:3.4.1**: the HDFS namenode and datanode of the air-gap simulation. It is configured entirely through environment variables (`CORE-SITE.XML...`, `HDFS-SITE.XML...`).
- **nvcr.io/nvidia/tritonserver:<tag>**: started by `deploy/scripts/modes_node.sh` on the GPU node for the Triton-mode runs. The script tries several tags and records the one that pulled.
- **nvidia/cuda:12.6.3-base-ubuntu22.04**: `docker run --rm --gpus all ... nvidia-smi` in the setup scripts, to prove the GPU is visible to containers before anything else starts.

3. What runs inside the containers

A container has one role, but the process tree inside it has several levels. These roles appear across all the compose files:

role	processes inside the container	listens on	image(s)	GPU
Spark master	<code>docker-init (tini) or bash -> java ...deploy.master.Master; tail -f</code> of its log keeps the container alive	7077 RPC, 8080 UI	multi-model-inference, ner-translate-worker, spark-lean	no
driver (runs inside the master container)	<code>docker exec ... python submit_*.py -> java SparkSubmit (Py4J gateway)</code>	4040 driver UI, a random RPC port, a BlockManager port	same as the master	tensor jobs: optional
Spark worker	<code>java ...deploy.worker.Worker -> one executor JVM per application -> pyspark.daemon -> forked Python workers</code>	a random worker port, 8081 UI	same image family as its master	GPU workers: yes
model server (kitchen)	<code>uvicorn (FastAPI)</code> with the models loaded once; <code>tesseract child</code> processes during OCR	8000	ner-translate-server	yes
HDFS namenode / datanode	<code>java NameNode / DataNode</code>	8020 RPC, 9870 WebHDFS / 9864, 9866	apache/hadoop	no
Triton	tritonserver	8000 HTTP, 8001 gRPC, 8002 metrics	tritonserver	yes
single dev container	<code>python benchmark/run_benchmark.py --mode all</code> (Spark in local mode)	4040, 8080	the image built by <code>docker-compose.yml</code>	yes

The Spark daemons are started with the scripts `start-master.sh / start-worker.sh` `spark://<master>:7077 -c <cores> -m <memory>`, which daemonise. The compose command then runs `tail -f` on the daemon's log so the container keeps running and `docker logs` shows Spark's output. A worker's `-c` and `-m` are what it offers the master. `spark.task.cpus = 2` (set by the platform) turns 4 cores into 2 task slots.

4. The compose files, one by one

docker-compose.yml - single development container

item	details
service / container	inference / multi-model-inference
image	built from <code>deploy/Dockerfile</code> (no <code>--target</code> , so the lean stage: see section 2.1)
runs	<code>python benchmark/run_benchmark.py --mode all</code> (every mode, Spark local mode)
mounts	the whole repository at <code>/app</code> (edit and re-run without rebuilding), <code>results/</code>
ports	4040 (driver UI), 8080
GPU	runtime: nvidia (legacy syntax; needs the nvidia runtime registered with Docker)
use when	a quick single-machine benchmark or a debugging session - no cluster

docker-compose.laptop.yml - Spark cluster sized for the 8 GB laptop

item	details
services	spark-master (+ driver, GPU visible); spark-gpu-worker (-c 4 -m 4g, GPU); spark-cpu-worker (CUDA hidden, 0 replicas, scale with --scale spark-cpu-worker=N)
image	multi-model-inference:latest (pre-built)
mounts	results, benchmark, inference, models, data, deploy, submit_job.py - current code without a rebuild
ports	7077, 8080, 4040
GPU	deploy.resources.reservations.devices (compose spec - works on Docker Desktop / WSL2)
used by	benchmark/run_campaign.sh (LEG=wsl2_docker), the execution-modes statistics, BYOM tests
use when	tensor models (built-ins, BYOM plugins) on the laptop, with RDD / UDF / GPU modes

docker-compose.cluster.yml - multi-worker cluster on Windows Docker Desktop

item	details
services	spark-master (CUDA hidden, driver memory 8g); 2 x spark-cpu-worker (-c 2 -m 4g); spark-gpu-worker (-c 4 -m 8g)
image	multi-model-inference:latest
mounts	results, benchmark, inference, models, data
ports	7077, 8080, 4040
GPU	none: the 'GPU' worker has CUDA_VISIBLE_DEVICES empty (a comment says to re-enable it after a torch rebuild for sm_120)
use when	CPU scaling experiments with several workers on one Windows machine; stop_cluster.ps1 tears it down

docker-compose.cluster.linux.yml - one Linux host / EC2, host networking

item	details
services	spark-master; 2 x spark-cpu-worker (-c 2 -m 4g); spark-gpu-worker (-c 4 -m 12g, runtime: nvidia)
image	multi-model-inference:latest
network	network_mode: host - the JVMs bind the host's own IP and ports; the workers join spark://localhost:7077, so single host only
mounts	none: runs the baked code, and results stay inside the containers (use docker cp)
use when	a single Linux GPU server; for several hosts use the docker run scripts (section 6)

docker-compose.ner_translate.yml - NER, dedicated cluster, in-process (Option A)

item	details
services	ner-translate-master, ner-translate-worker (-c 2 -m 4g) - both CUDA hidden, so NER runs on the CPU as configured
image	built from deploy/Dockerfile.ner_translate (compose names the images after the project: deploy-ner-translate-*)
mounts	results, data, models, wheels-hotfix/ner_translate -> /wheels-hotfix (applied at start)
ports	7078 -> 7077, 8081 -> 8080, 4041 -> 4040 (offset so it can run next to the shared cluster)
submit	docker exec -it ner-translate-master python submit_pipeline_job.py --pipeline ner_translate --input /app/data/ner_samples --partitions 2
use when	the executors must hold the models (no model server), isolated from the shared image's dependencies

docker-compose.ner_translate_server.yml - NER waiter / kitchen (Option B)

item	details
services	ner-translate-server (kitchen, GPU, :8001 -> 8000); ner-translate-master + ner-translate-worker (spark-lean, -c 4 -m 4g)
order	the worker waits until the master and the kitchen are healthy; the kitchen health check allows a 60 s model load
mounts	\$(NER_TRANSLATE_ROOT:-./models, data, results - start it with deploy/scripts/setup_ner_translate_server.sh, which checks the folders exist and sets NER_TRANSLATE_ROOT to an absolute path
ports	7080 -> 7077, 8083 -> 8080, 4043 -> 4040, 8001 -> 8000
GPU	the kitchen only, plus optional NVIDIA MPS (/tmp/nvidia-mps mounts, deploy/scripts/start_mps.sh on the host)
submit	same command as Option A; submit_pipeline_job.py sees service_url resolve and POSTs file paths to the kitchen
use when	GPU pipelines in production shape: one model copy per GPU, lean Spark nodes, the GPU shared by all tasks

docker-compose.airgap_sim.yml - air-gapped cluster simulation (project airgap-sim)

item	details
profiles	hdfs: sim-hdfs-namenode, sim-hdfs-datanode service: sim-ner-translate-server, sim-spark-master, sim-spark-worker cluster: sim-ner-cluster-master, sim-ner-cluster-worker (ner-translate-worker image, GPU)
model store	MODEL_STORE_URI = hdfs://hdfs-namenode:8020/models/weights, or file:///mnt/models (MODEL_FS_DIR mounted read-only), or unset (weights in the repository)
volumes	hdfs-name, hdfs-data (HDFS), kitchen-cache, worker-cache (the node-local model caches)
mounts	results, data, models, inference, submit_pipeline_job.py (current code); WEIGHTS_SRC -> /staging on the namenode
ports	9870 (WebHDFS UI), 8001 / 8083 / 4043 (service), 8084 / 4044 (cluster)
extras	init: true on the Python services (tini reaps OCR child processes); SPARK_DAEMON_MEMORY 256m; health checks on every service
use when	rehearsing the air-gapped deployment: models in HDFS or on the master, all 12 tests (deploy/airgap_sim_tests.ps1)

Topology of the air-gap simulation, the most complete of the files:

21. Air-gapped cluster simulation: models in HDFS, nodes fetch into a local cache

deploy/docker-compose.airgap_sim.yml - profiles hdfs / service / cluster; MODEL_STORE_URI selects where models come from

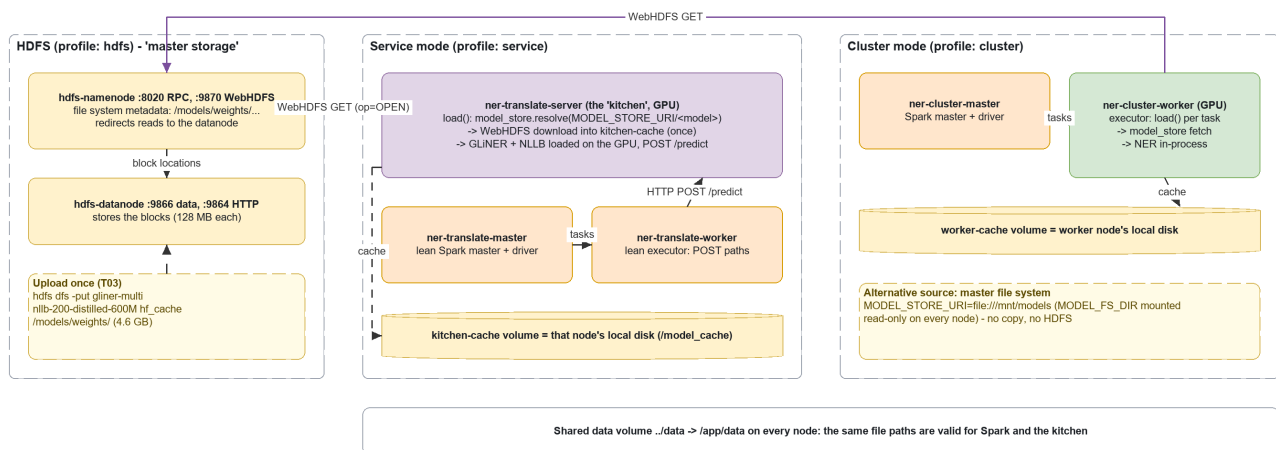


Figure 3 - docker-compose.airgap_sim.yml with its three profiles

5. How the containers work together

5.1 Networks and names

- Every compose project gets one bridge network, and service names become DNS names on it: spark-master, ner-translate-master, ner-translate-server, hdfs-namenode and so on. models/pipelines/manifest.json relies on this: master_url spark://ner-translate-master:7077 and service_url http://ner-translate-server:8000 are container names. --execution-mode auto uses the service only if that name resolves where the driver runs.
- Six of the seven files share one project.** A compose file without name: takes its project name from its folder, so every file in deploy/ is project deploy on network deploy_default. Containers started from different files can reach each other by name. docker compose up also warns about 'orphan containers' that belong to another file, and docker compose -f <file> down --remove-orphans removes the other files' containers too. The air-gap simulation sets name: airgap-sim and is isolated.
- host networking** (cluster.linux.yml and every docker run script on EC2 / the LAN) removes Docker's port mapping. Spark executors connect back to the driver on random ports, and the driver has to advertise an address the other hosts can reach (SPARK_LOCAL_IP in modes_node.sh). Across several machines that only works reliably with the hosts' real IPs.

5.2 Who talks to whom

from	to	protocol / port	carries
driver (in the master container)	Spark master	RPC :7077	application registration, executor requests
Spark master	Spark workers	RPC (worker port)	launch an executor for the application
driver	executors (in the worker containers)	RPC + BlockManager (random ports)	tasks, broadcast blocks, results
executor JVM	Python worker (same container)	localhost socket	pickled closures, path batches, results
Python worker (service mode)	kitchen	HTTP :8000 POST /predict	file paths in, entity JSON out
kitchen / Python worker	HDFS namenode -> datanode	WebHDFS :9870 -> :9864	model files, first load on a node only
Spark worker (Triton mode)	Triton	gRPC :8001	tensors in, predictions out
every container	bind mounts / named volumes	file system	documents, code, weights, results, model caches

5.3 Shared folders - the real integration layer

Spark moves file **names**; every container opens the files itself. So every container that reads documents or writes results must see the same folders at the same paths:

compose file	results	data	models	code (inference/, submit_*.py)	model caches
docker-compose.yml	yes	(whole repo)	(whole repo)	whole repo at /app	-
laptop	yes	yes	yes	inference/, benchmark/, deploy/, submit_job.py	-
cluster (Windows)	yes	yes	yes	inference/, benchmark/	-
cluster.linux	no	no	no	baked into the image	-
ner_translate	yes	yes	yes	baked (inference/, submit_pipeline_job.py not mounted)	-
ner_translate_server	yes	yes	yes (code + weights for the kitchen)	baked on the Spark side	-
airgap_sim	yes	yes	yes	inference/, submit_pipeline_job.py mounted	kitchen-cache, worker-cache volumes

Mounted code wins over baked code, so a change in the repository takes effect at the next container start without a rebuild. Where code is baked (cluster.linux, the Spark side of both NER files), the image has to be rebuilt, or the folder mounted, after a code change.

5.4 GPU access - three syntaxes

syntax	where	notes
deploy.resources.reservations.devices (driver: nvidia, capabilities: [gpu])	laptop, ner_translate_server, airgap_sim	compose spec; works with Docker Desktop / WSL2 and Linux + NVIDIA Container Toolkit
runtime: nvidia	docker-compose.yml, cluster.linux.yml	legacy; needs the nvidia runtime registered in the Docker daemon config
--gpus all	every docker run script	the docker run equivalent of the first
CUDA_VISIBLE_DEVICES= (empty)	cpu workers, cluster.yml, ner_translate.yml	hides the GPU from torch inside a container that could see it
SPARK_WORKER_OPTS gpu discovery script	modes_node.sh	Spark's own GPU scheduling (resource gpu, amount 1) for GPU-aware runs

5.5 Start order and health

Masters have an HTTP health check on the master UI (curl :8080); workers use depends_on: condition: service_healthy. In docker-compose.ner_translate_server.yml the worker also waits for the kitchen, whose health check passes only once uvicorn is serving, i.e. after the models have loaded (start_period: 60s). The air-gap simulation checks that /health reports ok, allows 60 retries for slow model loads, and uses init: true so exited OCR child processes are reaped. Without it a container could not be stopped ('PID is zombie').

6. Deployments without compose (docker run scripts)

Compose's service DNS works inside one Docker host. Multi-host clusters are started with plain `docker run --network host` on each machine, by these scripts:

script	runs on	containers it starts	image	notes
deploy/start_cluster.ps1, stop_cluster.ps1	Windows lab machines (WinRM for remote ones)	spark-master, spark-cpu-worker-*, spark-gpu-worker-*	multi-model-inference	IP list at the top of the script; image preloaded on every machine
deploy/run_lan_cluster_benchmarks.ps1 -> scripts/lan_cluster_benchmarks.sh	2-node LAN, Docker CE inside WSL2 (mirrored networking)	uses the running spark-master / GPU worker	multi-model-inference	docker exec via <code>ws1 -d <distro></code> , results by <code>docker cp</code>
deploy/scripts/setup_master.sh, setup_gpu_worker.sh, setup_and_run_gpu.sh	EC2 / Linux hosts	spark-master (+ cpu worker), spark-gpu-worker	multi-model-inference (built without <code>--target!</code>)	host network, <code>--gpus all</code> , <code>shm 4g</code>
deploy/run_aws_campaign.ps1 -> scripts/aws_campaign.sh	one g4dn (T4) node	spark-master + spark-gpu-worker	<code>--target final</code>	the benchmark campaign on AWS
deploy/run_aws_modes.ps1 -> scripts/modes_node.sh	CPU node (m5) + GPU node (g4dn)	master + cpu worker on the CPU node; gpu worker (Spark GPU discovery) + Triton on the GPU node	<code>--target final</code> + <code>tritonserver</code>	the execution-modes statistics (RDD / UDF / <code>predict_batch_udf</code> / Triton)
deploy/aws-cdk (spark_cluster_stack.py user data)	CDK-provisioned EC2	spark-master, spark-cpu-worker, spark-gpu-worker	docker load image.tar, or build (no <code>--target</code>)	<code>--restart unless-stopped</code>
deploy/run_gpu_benchmarks.ps1, run_benchmarks_cloud.ps1	EC2 via SSM / CloudFormation spot	master + cpu / gpu workers	multi-model-inference (build without <code>--target</code>)	older benchmark drivers
deploy/scripts/build_and_test_ner_translate_gpu.sh / _server.sh	a build machine with a GPU	short test containers	ner-translate-worker / spark-lean + ner-translate-server	build, smoke-test, then docker save for the air-gap transfer
deploy/scripts/setup_ner_translate_server.sh	the NER host	via <code>docker-compose.ner_translate_server.yml</code>	spark-lean + ner-translate-server	validates the folder layout first, then compose up

7. Which one to use when

30. Which compose file or script to use

Start from what you want to run; each leaf names the file and why

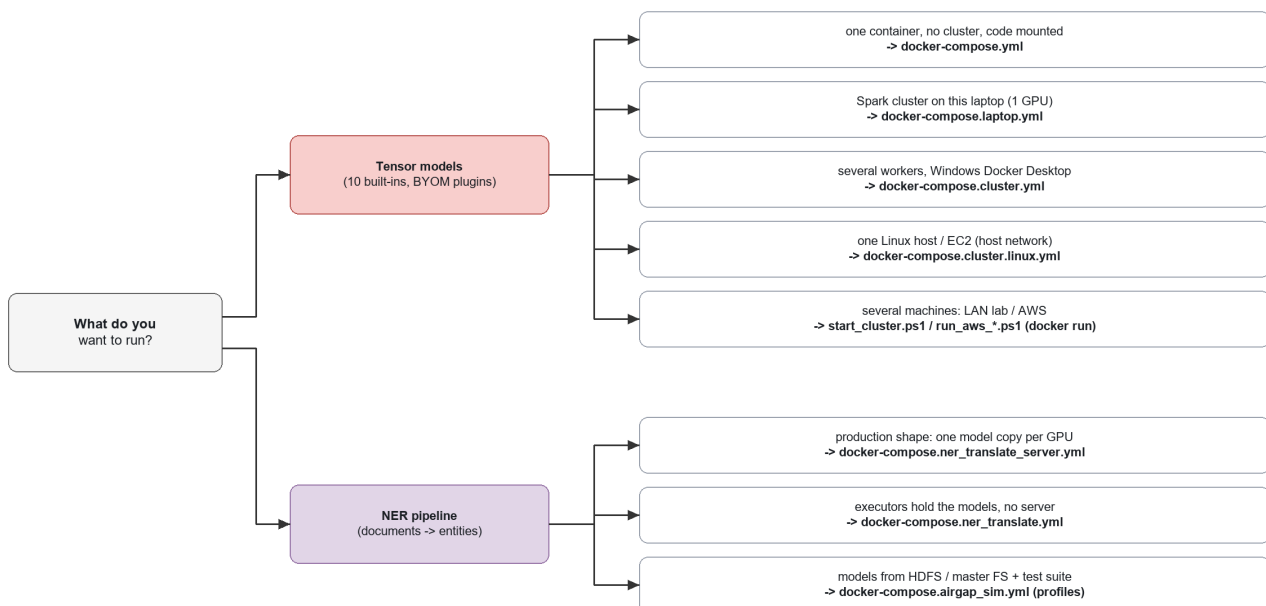


Figure 4 - choosing a compose file or script

7.1 What can run at the same time

combination	together?	why
laptop + cluster + cluster.linux + docker-compose.yml	no - one at a time	same host ports (8080, 4040, 7077) and the container name spark-master
ner_translate (A) + ner_translate_server (B)	no	both use the container name ner-translate-master (the file header says one at a time)
ner_translate_server + airgap_sim profile service	no	same host ports 8001, 8083, 4043
laptop + ner_translate (A)	yes (memory permitting)	ports offset (7078 / 8081 / 4041), different names; same project, so expect orphan warnings
laptop + airgap_sim	yes (memory permitting)	different ports and names, separate project and network
airgap_sim profiles hdfs + service + cluster	yes, but not on 6 GB	the tests stop the service profile before starting cluster: two model copies do not fit

7.2 Resource guide (this laptop: 8 GB RAM, 6 GB for Docker, GTX 1650 4 GB)

running	memory	GPU memory
kitchen (models loaded)	~1.3-1.9 GB RSS, more during the load	~3.0 GB (GLiNER fp16 + NLLB fp32)
spark-lean master + worker	~0.2 GB each JVM + executor	-
cluster-mode worker loading models	up to ~2.9 GB in the Python worker	~3.0 GB per task
HDFS namenode + datanode	256 MB heap each	-
multi-model-inference GPU worker (tensor models)	depends on the models and partitions	shared by the task slots

8. Build, save and move images (air gap)

```
# on a connected build machine (order matters: ner-translate-worker is FROM multi-model-inference)
docker build --target final -t multi-model-inference:latest -f deploy/Dockerfile .
docker build --target lean -t spark-lean:latest -f deploy/Dockerfile .
bash deploy/scripts/build_ner_translate_wheelhouse.sh # -> wheels/ner_translate
bash deploy/scripts/build_ner_translate_debs.sh # -> debs/ner_translate
docker build -t ner-translate-worker:latest -f deploy/Dockerfile.ner_translate .
docker build -t ner-translate-server:latest -f deploy/Dockerfile.ner_translate_server .
docker pull apache/hadoop:3.4.1

# move: save -> checksum -> media -> load on every node
docker save spark-lean:latest | gzip > spark-lean.tar.gz
docker save ner-translate-server:latest | gzip > ner-translate-server.tar.gz
sha256sum *.tar.gz > SHA256SUMS # verify on the other side: sha256sum -c SHA256SUMS
docker load -i spark-lean.tar.gz && docker load -i ner-translate-server.tar.gz
```

- What each node needs: for service mode (B), spark-lean on the Spark nodes and ner-translate-server on the GPU node. For cluster mode (A), ner-translate-worker on every Spark node. For tensor models, multi-model-inference on every node. HDFS nodes need apache/hadoop.
- The models do not travel in the images. They go into HDFS once (hdfs dfs -put gliner-multi nllb-200-distilled-600M hf_cache /models/weights/) or onto the master's shared folder.
- Code changes travel as a small bundle and are mounted into the containers (models/, inference/, submit_pipeline_job.py), so the multi-GB images only move when dependencies change. A single urgent dependency fix can go through wheels-hotfix (section 2.2).
- Offline rebuilds of ner-translate-worker need only the cached base image plus the wheelhouse, the .deb bundle and the code; --no-index and dpkg -i never contact a repository.

9. Gotchas

symptom	cause	fix
image tagged multi-model-inference has no torch	built without --target: the default stage is lean	build with --target final (or move the final stage to the end of the Dockerfile)
ModuleNotFoundError: No module named 'models' in a container	a bind mount pointed at a folder that does not exist; Docker mounts an empty one silently	start Option B with setup_ner_translate_server.sh (sets NER_TRANSLATE_ROOT); check with docker exec <c> ls /app/models
old code runs after an edit	that compose file bakes the code instead of mounting it	mount the folder (as airgap_sim does) or rebuild the image
'container name already in use' / port already allocated	two files with the same container_name or host ports (section 7.1)	docker compose -f <other file> down first
another file's containers vanish after down	same project deploy, and --remove-orphans	no --remove-orphans, or give each file its own project (-p or name:)
torch.cuda.is_available() is False	no GPU request in that service, or CUDA_VISIBLE_DEVICES is empty	use a GPU service / profile; check with docker run --rm --gpus all nvidia/cuda:12.6.3-base nvidia-smi
container cannot be stopped: 'PID is zombie'	OCR / tokenizer child processes not reaped	init: true on Python services
workers on other hosts cannot reach the driver	bridge networking across hosts	host networking + SPARK_LOCAL_IP (docker run scripts)
kitchen restarts or is killed while loading	Docker VM memory (6 GB on the laptop)	stop other containers; one model copy at a time
results missing on the host (cluster.linux)	results/ not mounted there	docker cp spark-master:/app/results .

10. Quick reference

```
# tensor models on the laptop
docker compose -f deploy/docker-compose.laptop.yml up -d [--scale spark-cpu-worker=2]
docker exec spark-master python submit_job.py --model resnet18 --samples 2000 \
  --mode gpu_only --master spark://spark-master:7077

# NER, waiter / kitchen (Option B)
bash deploy/scripts/setup_ner_translate_server.sh # validated compose up
docker exec ner-translate-master python submit_pipeline_job.py \
  --pipeline ner_translate --input /app/data/ner_samples --partitions 2

# NER, in-process (Option A)
docker compose -f deploy/docker-compose.ner_translate.yml up -d
docker exec ner-translate-master python submit_pipeline_job.py \
  --pipeline ner_translate --input /app/data/ner_samples --partitions 2

# air-gap simulation (PowerShell)
$env:MODEL_STORE_URI = "hdfs://hdfs-namenode:8020/models/weights"
docker compose -f deploy/docker-compose.airgap_sim.yml --profile hdfs --profile service up -d
.\deploy\airgap_sim_tests.ps1 # all 12 tests

# look inside any of them
docker compose -f <file> ps | logs -f <service> | exec <service> bash
docker compose -f <file> down # keeps named volumes; down -v deletes them (HDFS data, model caches)
```

Related documents: CLUSTER_RUNBOOK_MODES_AND_TESTS.pdf (every mode and test),
 NER_SPARK_DATA_PATHS_AND_EXECUTION.pdf (what moves between these containers, measured),
 NER_PIPELINE_END_TO_END_PROCESS.pdf (the process from build to stop).